prepare a complete SQL starting from creating table to trancastion controle all...

Complete SQL Tutorial: From Basics to Advanced

Table of Contents

1. Getting Started
2. Database & Table Creation
3. Data Types
4. Constraints
5. DML Operations (INSERT, UPDATE, DELETE)
6. DQL Operations (SELECT)
7. Filtering & Sorting
8. Aggregate Functions
9. GROUP BY & HAVING
10. JOINs
11. Subqueries
12. Views
13. Indexes
14. Transaction Control
15. Stored Procedures
16. Triggers

---

1. Getting Started

Starting MySQL

bash

```bash
# Command line
mysql -u root -p
```

```
3
4   # Or use MySQL Workbench/phpMyAdmin
```

Show Existing Databases

```sql
1   SHOW DATABASES;
```

---

2. Database & Table Creation

Create Database

```sql
1   -- Basic syntax
2   CREATE DATABASE database_name;
3
4   -- With condition (avoid error if exists)
5   CREATE DATABASE IF NOT EXISTS school;
6
7   -- Use database
8   USE school;
```

Create Table

```sql
1   -- Basic syntax
2   CREATE TABLE table_name (
3       column_name1 data_type constraints,
4       column_name2 data_type constraints,
5       ...
6   );
7
8   -- Example: Creating a Student table
9   CREATE TABLE students (
10      student_id INT PRIMARY KEY,
11      first_name VARCHAR(50) NOT NULL,
12      last_name VARCHAR(50) NOT NULL,
13      email VARCHAR(100) UNIQUE,
14      phone VARCHAR(15),
15      date_of_birth DATE,
16      gender CHAR(1),
17      marks DECIMAL(5,2),
18      city VARCHAR(30) DEFAULT 'Unknown',
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Example: Complete E-Commerce Database

sql

```sql
-- Create Database
CREATE DATABASE IF NOT EXISTS ecommerce;
USE ecommerce;

-- Categories Table
CREATE TABLE categories (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(100) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Products Table
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(200) NOT NULL,
    category_id INT,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT DEFAULT 0,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES
 categories(category_id)
        ON DELETE SET NULL ON UPDATE CASCADE
);

-- Customers Table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    address VARCHAR(255),
    city VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Orders Table
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2),
    status VARCHAR(20) DEFAULT 'Pending',
```

```
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Order Items Table
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE RESTRICT
);
```

---

3. Data Types

Numeric Data Types

sql

```
-- Integer types
TINYINT      -- -128 to 127 (or 0 to 255 unsigned)
SMALLINT     -- -32,768 to 32,767
MEDIUMINT    -- -8,388,608 to 8,388,607
INT          -- -2,147,483,648 to 2,147,483,647
BIGINT       -- Huge numbers

-- Decimal types
DECIMAL(5,2)    -- Total 5 digits, 2 after decimal (999.99)
FLOAT           -- Approximate decimal
DOUBLE          -- Double precision decimal

-- Examples
age TINYINT UNSIGNED          -- 0 to 255
salary DECIMAL(10,2)          -- 9999999.99
percentage FLOAT(3,2)         -- 99.99
```

String Data Types

sql

```
-- CHAR(n) - Fixed length (0-255)
gender CHAR(1)                -- 'M' or 'F'
```

```
    -- VARCHAR(n) - Variable length (0-65535)
    name VARCHAR(100)              -- 'John' uses 4 bytes

    -- TEXT types
    TINYTEXT     -- 255 bytes
    TEXT         -- 65,535 bytes
    MEDIUMTEXT   -- 16 MB
    LONGTEXT     -- 4 GB

    -- Example
    bio TEXT                       -- For long descriptions
```

Date/Time Data Types

sql

```
    DATE           -- 'YYYY-MM-DD' (2024-01-15)
    TIME           -- 'HH:MM:SS' (14:30:00)
    DATETIME       -- 'YYYY-MM-DD HH:MM:SS'
    TIMESTAMP      -- Unix timestamp
    YEAR           -- Year (2014)

    -- Example
    birth_date DATE
    created_at DATETIME
```

JSON Data Type (MySQL 5.7+)

sql

```
    -- JSON data type for storing JSON objects
    CREATE TABLE settings (
        id INT PRIMARY KEY,
        preferences JSON
    );

    INSERT INTO settings VALUES (1, '{"theme": "dark",
      "language": "en", "notifications": true}');
```

---

4. Constraints

NOT NULL

sql

```sql
-- Ensures column cannot have NULL value
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    salary DECIMAL(10,2)
);
```

UNIQUE

sql

```sql
-- All values must be different
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) UNIQUE,
    email VARCHAR(100) UNIQUE,
    phone VARCHAR(15) UNIQUE
);

-- Alternative syntax
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50),
    UNIQUE(username)
);
```

PRIMARY KEY

sql

```sql
-- Unique + NOT NULL (only ONE per table)
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    ...
);

-- Composite Primary Key (multiple columns)
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id)
);
```

FOREIGN KEY

sql

```sql
-- References another table's primary key
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    dept_id INT,
    name VARCHAR(100),
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON DELETE CASCADE    -- Delete employee when department deleted
        ON UPDATE CASCADE    -- Update employee when department ID updated
);

-- Different ON DELETE options:
-- CASCADE: Delete related rows
-- SET NULL: Set foreign key to NULL
-- RESTRICT: Prevent deletion
-- NO ACTION: Same as RESTRICT
```

DEFAULT

sql

```sql
-- Provides default value if none given
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2) DEFAULT 0.00,
    status VARCHAR(20) DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO products (product_id, name) VALUES (1, 'Laptop');
-- price will be 0.00, status will be 'Active'
```

CHECK

sql

```sql
-- Validates condition
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT CHECK (age >= 18),
    marks DECIMAL(5,2) CHECK (marks >= 0 AND marks <= 100),
    gender CHAR(1) CHECK (gender IN ('M', 'F', 'O'))
);

-- MySQL ignores CHECK constraint but syntax is accepted
```

AUTO_INCREMENT

```sql
-- Auto increment value
CREATE TABLE categories (
    cat_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50)
);

INSERT INTO categories (name) VALUES ('Electronics');
-- cat_id will be 1, 2, 3, ...

-- Start from specific number
ALTER TABLE categories AUTO_INCREMENT = 100;
```

---

5. DML Operations

INSERT - Single Row

```sql
-- Basic syntax
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);

-- Example
INSERT INTO students (student_id, first_name, last_name, email, marks, city)
VALUES (1, 'John', 'Smith', 'john@email.com', 85.5, 'New York');
```

INSERT - Multiple Rows

```sql
INSERT INTO students (student_id, first_name, last_name, email, marks, city)
VALUES
    (2, 'Sarah', 'Johnson', 'sarah@email.com', 92.0, 'Los Angeles'),
    (3, 'Mike', 'Brown', 'mike@email.com', 78.5, 'Chicago'),
```

```
    (4, 'Emily', 'Davis', 'emily@email.com', 88.0, 'New York'),
    (5, 'David', 'Wilson', 'david@email.com', 95.5, 'Boston');
```

INSERT - From Another Table

sql

```sql
-- Copy structure and data
CREATE TABLE top_students AS SELECT * FROM students WHERE marks > 90;

-- Copy specific columns
CREATE TABLE student_names AS
SELECT first_name, last_name FROM students;
```

UPDATE - Modify Data

sql

```sql
-- Basic syntax
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;

-- Update single column
UPDATE students
SET marks = 90
WHERE student_id = 1;

-- Update multiple columns
UPDATE students
SET marks = marks + 5, city = 'San Francisco'
WHERE city = 'New York';

-- Update with calculation
UPDATE products
SET price = price * 0.9  -- 10% discount
WHERE category_id = 1;

-- UPDATE with string functions
UPDATE students
SET first_name = UPPER(first_name)
WHERE student_id = 1;
```

DELETE - Remove Data

sql

```sql
-- Basic syntax
DELETE FROM table_name
WHERE condition;

-- Delete specific row
DELETE FROM students
WHERE student_id = 5;

-- Delete multiple rows
DELETE FROM students
WHERE marks < 35;

-- Delete alluse with caution!)
DELETE FROM students;
 rows (-- Or
TRUNCATE TABLE students;

-- Difference:
-- DELETE: Can use WHERE, can rollback (in transaction), slower
-- TRUNCATE: Cannot use WHERE, faster, resets auto_increment
```

---

6. DQL Operations

SELECT - Basic

sql

```sql
-- Select all columns
SELECT * FROM students;

-- Select specific columns
SELECT first_name, last_name, marks FROM students;

-- Select with aliases
SELECT first_name AS 'First Name',
       marks AS 'Score'
FROM students;

-- Select distinct values
SELECT DISTINCT city FROM students;
```

SELECT - With Calculations

sql

```sql
-- Arithmetic operations
SELECT product_name, price, price * 1.18 AS 'Price with Tax'
FROM products;

-- Concatenation
SELECT CONCAT(first_name, ' ', last_name) AS 'Full Name'
FROM students;
```

SELECT - With WHERE Conditions

sql

```sql
-- Basic comparison
SELECT * FROM students WHERE marks >= 80;
SELECT * FROM students WHERE city = 'New York';
SELECT * FROM students WHERE marks != 85;

-- Multiple conditions
SELECT * FROM students
WHERE marks >= 80 AND city = 'New York';

SELECT * FROM students
WHERE marks > 90 OR city = 'Boston';

-- IN operator
SELECT * FROM students
WHERE city IN ('New York', 'Boston', 'Chicago');

-- BETWEEN operator (inclusive)
SELECT * FROM students
WHERE marks BETWEEN 70 AND 90;

-- LIKE operator (pattern matching)
SELECT * FROM students WHERE first_name LIKE 'J%';      -- Starts with J
SELECT * FROM students WHERE email LIKE '%@gmail.com';   -- Ends with gmail.com
SELECT * FROM students WHERE name LIKE '%ohn%';          -- Contains 'ohn'

-- Wildcards: % (any chars), _ (single char)
```

---

7. Filtering & Sorting

ORDER BY

sql

```sql
-- Ascending (default)
SELECT * FROM students ORDER BY marks;

-- Descending
SELECT * FROM students ORDER BY marks DESC;

-- Multiple columns
SELECT * FROM students
ORDER BY city ASC, marks DESC;

-- Order by expression
SELECT * FROM students
ORDER BY (marks * 1.1) DESC;  -- 10% bonus marks
```

LIMIT

sql

```sql
-- Top N rows
SELECT * FROM students ORDER BY marks DESC LIMIT 3;

-- With offset (skip first 2, then get 3)
SELECT * FROM students ORDER BY marks DESC LIMIT 2, 3;

-- Pagination example (page 2, page size 10)
SELECT * FROM students LIMIT 10 OFFSET 10;  -- Page 2
```

Combined Example

sql

```sql
SELECT first_name, last_name, marks, city
FROM students
WHERE marks >= 70
ORDER BY marks DESC
LIMIT 5;
```

---

8. Aggregate Functions

COUNT

sql

```sql
-- Count all rows
SELECT COUNT(*) FROM students;

-- Count non-null values
SELECT COUNT(email) FROM students;

-- Count distinct values
SELECT COUNT(DISTINCT city) FROM students;
```

SUM & AVG

```sql
-- Sum
SELECT SUM(marks) FROM students;
SELECT SUM(marks) FROM students WHERE city = 'New York';

-- Average
SELECT AVG(marks) FROM students;
SELECT AVG(price) FROM products WHERE category_id = 1;
```

MIN & MAX

```sql
-- Minimum
SELECT MIN(marks) FROM students;
SELECT MIN(price) FROM products;

-- Maximum
SELECT MAX(marks) FROM students;
SELECT product_name, MAX(price) FROM products;
```

Using with GROUP BY

```sql
-- Average marks by city
SELECT city, AVG(marks) AS 'Average Marks'
FROM students
GROUP BY city;

-- Count by category
SELECT category_id, COUNT(*) AS 'Product Count'
FROM products
GROUP BY category_id;

-- Total sales by customer
SELECT customer_id, SUM(total_amount) AS 'Total Spent'
```

```
FROM orders
GROUP BY customer_id;
```

---

9. GROUP BY & HAVING

GROUP BY

sql

```sql
-- Basic grouping
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city;

-- Multiple columns
SELECT city, gender, COUNT(*)
FROM students
GROUP BY city, gender;

-- With aggregate function
SELECT
    category_id,
    COUNT(*) AS total_products,
    AVG(price) AS avg_price,
    MIN(price) AS min_price,
    MAX(price) AS max_price
FROM products
GROUP BY category_id;
```

HAVING

sql

```sql
-- Filter groups (after grouping)
SELECT city, COUNT(*) AS cnt
FROM students
GROUP BY city
HAVING cnt >= 2;

-- Having vs Where
-- WHERE: filters rows before grouping
-- HAVING: filters groups after grouping

-- Example: Cities with average marks > 80
SELECT city, AVG(marks) AS avg_marks
FROM students
GROUP BY city
```

```sql
HAVING avg_marks > 80;

-- Combined
SELECT city, AVG(marks) AS avg_marks
FROM students
WHERE marks >= 40
GROUP BY city
HAVING avg_marks > 70
ORDER BY avg_marks DESC;
```

Complete Example with All Clauses

sql

```sql
SELECT
    city,
    gender,
    COUNT(*) AS total_students,
    ROUND(AVG(marks), 2) AS avg_marks,
    MAX(marks) AS highest_marks,
    MIN(marks) AS lowest_marks
FROM students
WHERE date_of_birth >= '2000-01-01'
GROUP BY city, gender
HAVING COUNT(*) >= 1
ORDER BY avg_marks DESC
LIMIT 10;
```

---

10. JOINs

Sample Tables for JOINs

sql

```sql
-- Departments
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

INSERT INTO departments VALUES
(1, 'HR'), (2, 'IT'), (3, 'Sales'), (4, 'Marketing');

-- Employees
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
```

```
    salary DECIMAL(10,2)
);

INSERT INTO employees VALUES
(1, 'John', 1, 50000),
(2, 'Sarah', 2, 75000),
(3, 'Mike', 2, 80000),
(4, 'Emma', 3, 60000),
(5, 'David', NULL, 55000);  -- No department
```

INNER JOIN

sql

```
-- Returns only matching rows
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- Equivalent to
SELECT e.emp_name, d.dept_name
FROM employees e, departments d
WHERE e.dept_id = d.dept_id;
```

LEFT JOIN (LEFT OUTER JOIN)

sql

```
-- All rows from left table + matching from right
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- Result: David (no department) will show NULL for
  dept_name
```

RIGHT JOIN (RIGHT OUTER JOIN)

sql

```
-- All rows from right table + matching from left
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- Result: Marketing (no employees) will show NULL for
  emp_name
```

FULL OUTER JOIN

```sql
-- MySQL doesn't support FULL OUTER JOIN directly
-- Use UNION

SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

SELF JOIN

```sql
-- Join table with itself
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    manager_id INT
);

-- Find employees and their managers
SELECT
    e.name AS Employee,
    m.name AS Manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

CROSS JOIN

```sql
-- Cartesian product (all combinations)
SELECT * FROM employees CROSS JOIN departments;
```

Multiple JOINs

```sql
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    oi.quantity,
```

```
    oi.unit_price
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id;
```

---

11. Subqueries

Subquery in WHERE

sql

```sql
-- Find students with above average marks
SELECT * FROM students
WHERE marks > (SELECT AVG(marks) FROM students);

-- Find products priced higher than average
SELECT * FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Using IN with subquery
SELECT * FROM students
WHERE city IN (
    SELECT DISTINCT city
    FROM students
    WHERE marks > 90
);

-- Correlated subquery (depends on outer query)
SELECT * FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE dept_id = e.dept_id
);
```

Subquery in FROM

sql

```sql
-- Derived table
SELECT * FROM (
    SELECT city, AVG(marks) AS avg_marks
    FROM students
    GROUP BY city
```

```
) AS city_avg
WHERE avg_marks > 80;
```

Subquery in SELECT

sql

```sql
-- Scalar subquery
SELECT
    student_name,
    marks,
    (SELECT AVG(marks) FROM students) AS class_avg,
    marks - (SELECT AVG(marks) FROM students) AS diff_from_avg
FROM students;
```

EXISTS / NOT EXISTS

sql

```sql
-- Find customers who have placed orders
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

-- Find products never ordered
SELECT * FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);
```

Common Table Expression (CTE)

sql

```sql
-- Using WITH clause (MySQL 8.0+)
WITH city_avg AS (
    SELECT city, AVG(marks) AS avg_marks
    FROM students
    GROUP BY city
)
SELECT * FROM city_avg WHERE avg_marks > 80;

-- Multiple CTEs
WITH
    high_scorers AS (
        SELECT * FROM students WHERE marks > 90
```

```
    ),
    city_counts AS (
        SELECT city, COUNT(*) AS cnt FROM students GROUP BY city
    )
SELECT h.*, c.cnt
FROM high_scorers h
JOIN city_counts c ON h.city = c.city;
```

---

12. Views

Create View

sql

```sql
-- Basic view
CREATE VIEW active_students AS
SELECT student_id, first_name, last_name, marks
FROM students
WHERE marks >= 40;

-- Complex view with joins
CREATE VIEW order_summary AS
SELECT
    o.order_id,
    c.customer_name,
    o.order_date,
    o.status,
    o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;

-- Using view
SELECT * FROM active_students;
SELECT * FROM order_summary WHERE status = 'Pending';
```

Update View

sql

```sql
-- Updatable view (simple, single table)
CREATE VIEW simple_students AS
SELECT student_id, first_name, marks FROM students;

-- Insert through view
INSERT INTO simple_students VALUES (101, 'New Student', 75);
```

```sql
-- This inserts into underlying table!

-- Drop view
DROP VIEW active_students;
```

With CHECK Option

```sql
CREATE VIEW senior_students AS
SELECT * FROM students WHERE marks >= 60
WITH CHECK OPTION;

-- This will fail: marks < 60
INSERT INTO senior_students (student_id, first_name, marks)
VALUES (102, 'Test', 50);
-- Error: WITH CHECK OPTION violation
```

---

13. Indexes

Create Index

```sql
-- Basic index
CREATE INDEX idx_city ON students(city);

-- Composite index
CREATE INDEX idx_name_city ON students(first_name,
 last_name);

-- Unique index
CREATE UNIQUE INDEX idx_email ON customers(email);

-- Full-text index (for text search)
ALTER TABLE products ADD FULLTEXT(product_name,
 description);
```

View Indexes

```sql
SHOW INDEX FROM students;
-- Or
SHOW INDEX FROM students\G
```

Drop Index

```sql
DROP INDEX idx_city ON students;
```

When to Use Indexes

- ✅ ON columns used in WHERE, JOIN, ORDER BY
- ✅ On small tables (overhead)
- ❌ On columns with low cardinality (yes/no)
- ❌ On frequently updated columns

---

14. Transaction Control

What is a Transaction?

A transaction is a sequence of SQL operations that are treated as a single unit of work.

ACID Properties

- **Atomicity**: All succeed or all fail
- **Consistency**: Database remains consistent
- **Isolation**: Concurrent transactions don't interfere
- **Durability**: Changes are permanent once committed

Basic Transaction Commands

```sql
-- Start transaction
START TRANSACTION;

-- Or
BEGIN;

-- Your SQL operations
INSERT INTO accounts (id, balance) VALUES (1, 1000);
INSERT INTO accounts (id, balance) VALUES (2, 500);

-- Make changes permanent
COMMIT;

```

```sql
-- Undo all changes since transaction started
ROLLBACK;
```

Example: Money Transfer

sql

```sql
START TRANSACTION;

-- Deduct from sender
UPDATE accounts
SET balance = balance - 500
WHERE id = 1;

-- Add to receiver
UPDATE accounts
SET balance = balance + 500
WHERE id = 2;

-- Check if both operations succeeded
-- If error, rollback
-- Otherwise commit
COMMIT;
```

SAVEPOINT

sql

```sql
BEGIN;

INSERT INTO students (student_id, name) VALUES (1, 'John');
SAVEPOINT point1;

INSERT INTO students (student_id, name) VALUES (2,
  'Sarah');
SAVEPOINT point2;

INSERT INTO students (student_id, name) VALUES (3, 'Mike');
-- Error! Duplicate key

ROLLBACK TO SAVEPOINT point2;  -- Keeps point1 and point2
  insertions
-- Rollback to point1: keeps only first insertion
-- ROLLBACK: removes all
```

Transaction Isolation Levels

sql

```sql
-- View current level
SELECT @@transaction_isolation;

-- Set isolation level
SET TRANSACTION ISOLATION LEVEL
    READ UNCOMMITTED
    -- or READ COMMITTED
    -- or REPEATABLE READ (default MySQL)
    -- or SERIALIZABLE
```

Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	✅	✅	✅
READ COMMITTED	❌	✅	✅
REPEATABLE READ	❌	❌	✅ (MySQL: ❌)
SERIALIZABLE	❌	❌	❌

AUTOCOMMIT

```sql
-- Check autocommit mode
SELECT @@autocommit;

-- Disable autocommit (manual commit required)
SET autocommit = 0;

-- After operations
COMMIT;  -- or ROLLBACK
```

---

15. Stored Procedures

Create Procedure

```sql
-- Simple procedure
DELIMITER //

CREATE PROCEDURE get_all_students()
BEGIN
    SELECT * FROM students;
END //

```

```sql
DELIMITER ;

-- Call procedure
CALL get_all_students();
```

Procedure with Parameters

```sql
DELIMITER //

CREATE PROCEDURE get_students_by_city(
    IN city_name VARCHAR(50)
)
BEGIN
    SELECT * FROM students
    WHERE city = city_name;
END //

DELIMITER ;

-- Call
CALL get_students_by_city('New York');
```

Procedure with IN, OUT, INOUT

```sql
DELIMITER //

CREATE PROCEDURE get_student_count(
    IN city_name VARCHAR(50),
    OUT count INT
)
BEGIN
    SELECT COUNT(*) INTO count
    FROM students
    WHERE city = city_name;
END //

DELIMITER ;

-- Call with output variable
CALL get_student_count('New York', @cnt);
SELECT @cnt;
```

Procedure with Logic

```sql
```

```sql
DELIMITER //

CREATE PROCEDURE update_student_marks(
    IN student_id INT,
    IN new_marks DECIMAL(5,2)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error occurred' AS message;
    END;

    START TRANSACTION;

    UPDATE students
    SET marks = new_marks
    WHERE student_id = student_id;

    COMMIT;

    SELECT 'Marks updated successfully' AS message;
END //

DELIMITER ;
```

Drop Procedure

```sql
DROP PROCEDURE get_all_students;
```

---

16. Triggers

Create Trigger

```sql
-- Trigger before INSERT
DELIMITER //

CREATE TRIGGER before_student_insert
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
    SET NEW.created_at = NOW();
    IF NEW.marks < 0 THEN
```

```
            SET NEW.marks = 0;
        END IF;
    END //

    DELIMITER ;
```

Trigger after INSERT/UPDATE/DELETE

sql

```
    -- Audit log trigger
    CREATE TABLE student_audit (
        audit_id INT PRIMARY KEY AUTO_INCREMENT,
        student_id INT
```